

PREMIUM COURSE BOOK

React Hooks & Full Transcript & Important Words

Complete guide & 5 chapters & Real words from course & Return anytime &
Print-ready A4 with real margins, page numbers, headers/footers, TOC
with real page numbers

5 Chapters

3 min

7 Key Words

Table of Contents

Professional book ½ 5 chapters ½ Real page numbers from final layout ½ A4 print-ready

CHAPTER 1

Introduction ½ What Are React Hooks? 3

CHAPTER 2

useState ½ Exact Syntax and Array Return 4

CHAPTER 3

Asynchronous State ½ The setState Trap 5

CHAPTER 4

Stale Closure Bug ½ Real Example and Fix 6

CHAPTER 5

useEffect ½ Empty Deps and Cleanup 7

CHAPTER 1

[00:00]

Introduction ï¿½ What Are React Hooks?

Hey everyone, welcome back. Today we're diving deep into React Hooks - specifically `useState` and `useEffect`. This is going to be super practical.

So what are Hooks? The React docs say: "Hooks are functions that let you hook into React state and lifecycle features from function components." That's the exact definition. Before Hooks, we had to use class components for state. Now with function components, we can do everything. This chapter covers the fundamental definition and why Hooks matter.



CHAPTER 2

[02:15]

useState içi 1/2 Exact Syntax and Array Return

Let's start with useState. The exact syntax is: `const [count, setCount] = useState(0)`. This is important - useState returns an array with two elements.

First element is the current state value - count. Second is setter function - setCount. The 0 in useState(0) is initial value. This is crucial to understand. The array destructuring is not optional içi 1/2 it's how useState is designed. Many beginners try to use object destructuring and fail.



CHAPTER 3

[03:30]

Asynchronous State ½ The setState Trap

Now here's something important that trips people up: `setCount` doesn't update count immediately. It's asynchronous. The docs say "setState is asynchronous".

Let me show you. If you do `setCount(count + 1)` and then `console.log(count)`, you'll still see old value. Because React batches updates. This is one of the most common bugs in React. People expect immediate update, but React batches for performance.

So how do you get new value? Use functional update: `setCount(prev => prev + 1)`. This guarantees you get latest. Functional updates are essential when new state depends on old state.



CHAPTER 4

[06:10]

Stale Closure Bug ١٢½ Real Example and Fix

Now, big one - stale closure. This is exact phrase from video: "stale closure bug". When you use count inside setInterval with empty deps, it captures count = 0 forever.

The code: `useEffect(() => { setInterval(() => setCount(count + 1), 1000) }, [])`. This is bug - count stuck at 0. Because closure captures count at time of effect creation. Empty deps means effect runs once, captures initial count = 0, and never updates.

Fix is: `useEffect(() => { setInterval(() => setCount(c => c + 1), 1000) }, [])`. Use functional update. Functional update doesn't need count in closure, it receives latest value as argument.



CHAPTER 5

[08:30]

useEffect 1½ Empty Deps and Cleanup

Next, `useEffect`. Exact words: "useEffect with empty dependency array runs once, like `componentDidMount`".

Syntax: `useEffect(() => { console.log('runs once') }, [])`. Empty array means run once. This is equivalent to `componentDidMount` in class components. But you must cleanup. The video says: "Always cleanup in `useEffect` return". Example: `return () => clearInterval(timer)`. Cleanup is crucial to avoid memory leaks. If you set interval, timeout, or subscription, always clean it up in return function.

